Functional programming Mini Project - Server Log Analyzer

Group Members

- EG/2020/3990 - Jayasooriya LPM
- EG/2020/3994 - Jayathilake HACP
- EG/2020/3996 - Jayawardhana MVTI
- EG/2020/4040 - Lakpahana AGS

Technology Stack: Haskell (GHC 9.x), Stack Build Tool
GitHub Repo: https://github.com/ChandulaJ/haskell-log-analyzer
Dataset: https://www.kaggle.com/datasets/eliasdabbas/web-server-access-logs

1. Problem Statement and Industrial Motivation

1.1 Real-World Context

Modern web infrastructure generates **massive volumes of log data** daily. Enterprise servers handling millions of requests produce log files that can exceed gigabytes in size within hours. These logs contain critical operational intelligence about:

- **Traffic patterns** - User behavior, peak usage times, and popular endpoints
- **Security threats** - DDoS attacks, bot traffic, and unauthorized access attempts
- **System performance** - Error rates, response times, and resource bottlenecks
- **Compliance requirements** - Audit trails for regulatory standards

1.2 Industrial Challenges

Traditional imperative log analysis systems face significant limitations:

1. **Data Integrity Issues**: Mutable state in concurrent processing leads to race conditions and inconsistent results
2. **Scalability Constraints**: Difficulty parallelizing operations safely without complex locking mechanisms
3. **Reliability Concerns**: Side effects make testing difficult and results unpredictable
4. **Maintenance Complexity**: Tightly coupled code with side effects throughout makes modifications risky

1.3 Functional Solution

- **Immutable Data Structures**: Guarantees thread-safety and reproducible results
- **Parallel Processing**: Leverages Haskell's parallel evaluation strategies for safe concurrency
- **Type Safety**: Compile-time guarantees eliminate entire classes of runtime errors
- **Compositional Architecture**: Modular pipeline design enables easy testing and extension

The analyzer processes Apache/Nginx Combined Log Format, extracting metrics such as traffic patterns, error distributions, anomaly detection, and user session analysis.

2. Functional Design

2.1 System Architecture

The system follows a **functional pipeline architecture** with clear separation between pure logic and effects:

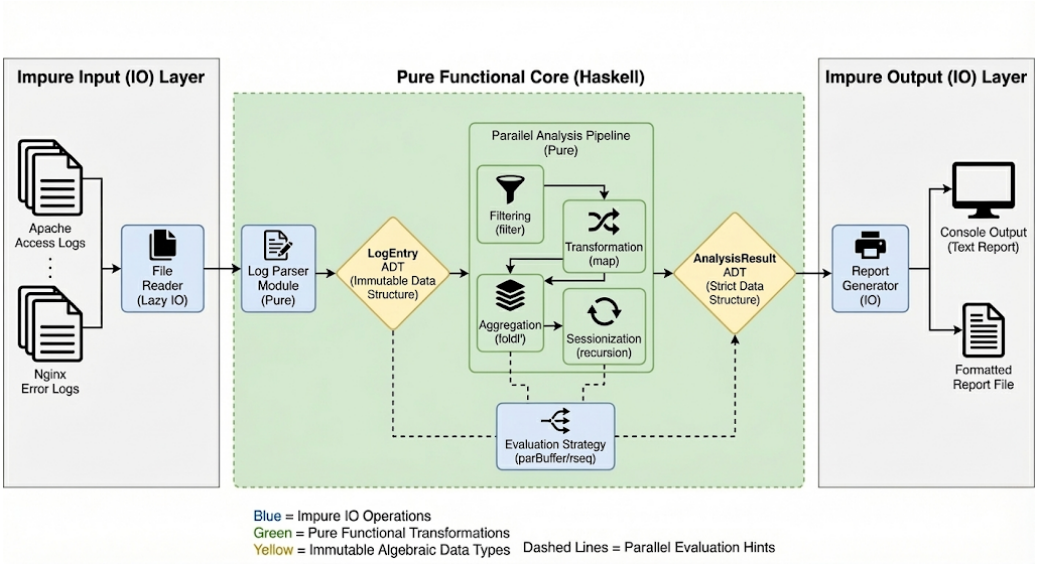

Figure 1: The figure of System Architecture

This architecture isolates side effects at the boundaries while keeping core transformations pure and testable.

2.2 Core Data Types (ADTs)

HTTP Method Representation (Sum Type)

Purpose: Type-safe representation of HTTP methods with extensibility for unknown methods.

Log Entry Structure (Product Type)

Purpose: Structured representation of parsed log entries with strong typing.

Status Categorization (Sum Type)

Purpose: Pattern-based classification of HTTP status codes.

Analysis Result (Product Type with Strict Fields)

Purpose: Aggregated analysis results with strict evaluation for parallel processing.

2.3 Main Functions with Type Signatures

Parsing Functions

```
-- Parse a single log line using regex pattern matching
parseLogLine :: String -> Maybe LogEntry
```

```haskell
-- Parse entire file content into list of log entries
parseLogFile :: String -> [LogEntry]

-- Parse with statistics tracking
parseLogFileWithStats :: String -> ([LogEntry], Int, Int)

-- Parse timestamp in Apache format
parseTimestamp :: String -> Maybe UTCTime

-- Parse HTTP method from string
parseMethod :: String -> Maybe HttpMethod
```

Processing Functions

```haskell
-- Count entries by status category using strict fold
countByLevel :: [LogEntry] -> Map StatusCategory Int

-- Extract top N IP addresses by request count
topIPs :: Int -> [LogEntry] -> [(String, Int)]

-- Bucket errors into time intervals
errorsOverTime :: NominalDiffTime -> [LogEntry] -> [(UTCTime, Int)]

-- Group requests into sessions with timeout-based splitting
sessionize :: NominalDiffTime -> [LogEntry] -> [[LogEntry]]

-- Detect traffic and error anomalies
computeAnomalies :: [LogEntry] -> [String]

-- Main processing pipeline combining all analyses
processLogs :: [LogEntry] -> AnalysisResult
```

I/O Functions

```haskell
-- Read log file with error handling
readLogFile :: FilePath -> IO String

-- Output formatted analysis report
writeReport :: AnalysisResult -> Int -> Int -> IO ()
```

Utility Functions

```haskell
-- Bucket timestamp to nearest interval boundary
bucketTime :: NominalDiffTime -> UTCTime -> UTCTime

-- Extract domain from referrer URL
extractDomain :: String -> String

-- Classify user agent into bot types
classifyBot :: String -> BotType
```

3. Functional Programming Concepts Demonstrated

3.1 Pure Functions

All data transformations are pure - they have no side effects, always produce the same output for the same input, and don't modify external state.

Example: Status categorization: Predictable, cacheable, testable, and safe for parallel execution.

3.2 Algebraic Data Types (ADTs)

Sum types (OR relationships) and **product types** (AND relationships) model the domain precisely. Compiler enforces all cases are handled (exhaustiveness checking), prevents invalid states.

3.3 Recursion and Tail Recursion

Natural iteration through recursive patterns without mutable loop variables:

```haskell
-- Session splitting using recursion with accumulator
splitSessions :: [LogEntry] -> [[LogEntry]]
splitSessions [] = []
splitSessions (x:xs) = go [x] xs
  where
    go currentSession [] = [reverse currentSession]
    go currentSession@(latest:_) (next:rest)
      | gap > timeout = reverse currentSession : go [next] rest
      | otherwise     = go (next:currentSession) rest
      where
        gap = diffUTCTime (leTimestamp next) (leTimestamp latest)
```

Benefits: Clear logic, no index management, compiler can optimize tail calls.

3.4 Higher-Order Functions

Functions as first-class values enable powerful abstractions:

```haskell
-- Generic aggregator accepting an extractor function
requestsPerEndpoint :: (LogEntry -> Maybe String) -> [LogEntry] -> Map String Int
requestsPerEndpoint extractor = foldl' aggregate M.empty
  where
    aggregate acc entry = case extractor entry of
      Just endpoint -> M.insertWith (+) endpoint 1 acc
      Nothing -> acc
```

Common higher-order patterns used,

map Transform each element (e.g., map parseLogLine lines)

filter: Select elements (e.g., filter (isError . leStatusCode))

foldl': Accumulate results with strict evaluation (e.g., building Maps)

sortOn: Sort by extracted key (e.g., sortOn leTimestamp)

Benefits: Eliminates code duplication, creates reusable abstractions.

3.5 Lazy Evaluation

Computation is deferred until results are needed, enabling infinite data structures and memory efficiency:

```
parseLogFile :: String -> [LogEntry]
parseLogFile content =
  [entry | Just entry <- map parseLogLine (lines content)]
```

Only the needed log entries are parsed and held in memory at any time, allowing processing of files larger than available RAM.

Benefits: Memory-efficient streaming, ability to work with infinite sequences.

3.6 Immutability

No mutable state eliminates entire classes of bugs:

```
countByLevel :: [LogEntry] -> Map StatusCategory Int
countByLevel = foldl' countEntry M.empty
  where
    countEntry acc entry =
      let category = statusCategory (leStatusCode entry)
      in M.insertWith (+) category 1 acc  -- Creates new map
```

Each operation creates a new data structure rather than modifying existing ones.

Benefits: Thread-safe by default, easier reasoning about code, time-travel debugging.

3.7 Parallel and Concurrent Processing

Safe parallelism without locks using evaluation strategies:

```
sessionize :: NominalDiffTime -> [LogEntry] -> [[LogEntry]]
sessionize timeout entries =
  concatMap sessionizeIP groupedByIP `using` parBuffer 100 rseq
```

The using function applies parallel evaluation strategy parBuffer 100 rseq, which evaluates up to 100 elements in parallel. Since all functions are pure, this is completely safe.

Benefits: Automatic parallelization, no race conditions, scales to multiple cores.

3.8 Monads for Effect Management

Maybe Monad for composable error handling:

```
parseLogLine :: String -> Maybe LogEntry
parseLogLine line = do
  -- Pattern match and extract fields
  timestamp <- parseTimestamp timeStr  -- Fails gracefully if invalid
```

```
  method <- parseMethod methodStr
  status <- readMaybe statusStr
  return LogEntry {...}
```

IO Monad for controlled side effects:

```
main :: IO ()
main = do
  filePath <- getFilePath        -- IO: User interaction
  content <- readLogFile filePath -- IO: File reading
  let entries = parseLogFile content   -- Pure: Parsing
  let result = processLogs entries     -- Pure: Processing
  writeReport result             -- IO: Output
```

Benefits: Eliminates null pointer exceptions, clear separation of pure/impure code.

3.9 Type-Driven Development

Type signatures as documentation and contracts:

```
topIPs :: Int -> [LogEntry] -> [(String, Int)]
errorsOverTime :: NominalDiffTime -> [LogEntry] -> [(UTCTime, Int)]
sessionize :: NominalDiffTime -> [LogEntry] -> [[LogEntry]]
```

The type signature topIPs :: Int -> [LogEntry] -> [(String, Int)] clearly communicates: “Give me a number and a list of log entries, and I’ll return IP addresses paired with counts.”

Benefits: Self-documenting code, compiler enforces correctness, guides implementation.

3.10 Pattern Matching

Destructure data elegantly:

```
parseMethod :: String -> Maybe HttpMethod
parseMethod "GET"     = Just GET
parseMethod "POST"    = Just POST
parseMethod "DELETE"  = Just DELETE
parseMethod other     = Just (UNKNOWN other)
```

Benefits: Exhaustiveness checking ensures all cases handled, readable code.

4. Expected Outputs and Possible Extensions

4.1 Current Output

The analyzer produces a comprehensive report including:

```
==========================================
     LOG ANALYSIS REPORT
==========================================
Total Lines Processed: 21
Successful Parses:     20
Failed Parses:         1
```

```
-------------------------------------------
Traffic by Status Category:
  Success         : 19
  ClientError     : 1

Top 10 IP Addresses:
  40.77.167.129   : 11 requests
  31.56.96.51     : 2 requests
  ...

Error Distribution (Hourly Buckets):
  2019-01-22 00:00:00 UTC : 1 errors
===========================================

!!! ANOMALIES DETECTED !!!
  [ALERT] High traffic from IP 40.77.167.129: 11 requests
```

4.2 Possible Extensions

1. Real-time Stream Processing

- Monitor logs in real-time using conduit or pipes libraries
- Trigger alerts immediately when anomalies detected
- Integrate with monitoring systems (Prometheus, Grafana)

2. Advanced Anomaly Detection

Machine learning integration for anomaly scoring

- Statistical outlier detection (Z-score, IQR methods)
- Pattern recognition for attack signatures
- Behavioral analysis for bot detection

3. Geolocation Analysis

IP to location mapping

- Integrate with GeoIP databases
- Visualize global traffic distribution
- Detect geographic attack patterns

4. Performance Metrics

Response time analysis

- Calculate percentiles (p50, p95, p99)
- Identify performance bottlenecks
- SLA compliance monitoring

5. Why Functional Programming Improves Reliability and Concurrency

5.1 Reliability Advantages

1. Referential Transparency

Pure functions always produce the same output for the same input:

```
statusCategory 404 == statusCategory 404  -- Always true
```

This enables, **Deterministic testing**: Same inputs always yield same results, **Memoization**: Safely cache function results, **Equational reasoning**: Substitute function calls with their values

2. Type Safety

The compiler prevents entire classes of errors:

Real-world impact: Zero null pointer exceptions, zero type coercion bugs.

3. Immutability Guarantees

Once data is created, it cannot be modified:

Benefits: - No defensive copying needed - Easier debugging (data doesn't change under you) Safe sharing between threads

4. Explicit Effect Management

The type system distinguishes pure from impure code:

This makes code behavior **explicit and auditable**.

5.2 Concurrency Advantages

1. Fearless Parallelism

Pure functions can be parallelized without coordination,

```
-- Process each IP's sessions in parallel
sessionize timeout entries =
  concatMap sessionizeIP groupedByIP `using` parBuffer 100 rseq
```

No locks, mutexes, or semaphores needed. The compiler guarantees thread safety.

2. No Race Conditions

Race conditions are **impossible** with immutable data:

```
-- Both threads safely read the same entries
let thread1 = processLogs entries
let thread2 = computeAnomalies entries
```

In imperative languages, this would require careful locking to prevent data corruption.

3. Scalability

The log analyzer automatically scales to available CPU cores

No code changes needed, just compile with parallel runtime flags.

4. Automatic Work Distribution

Haskell's parallel strategies handle work distribution:

```
parBuffer 100 rseq  -- Evaluates up to 100 elements in parallel
parList rdeepseq    -- Fully evaluates list in parallel
```

The runtime system manages thread pools, work stealing, and load balancing.

5.3 Industrial Use Cases

DevOps & Site Reliability Engineering

- **Incident Response**: Quickly identify error spikes and affected systems
- **Capacity Planning**: Analyze traffic trends for infrastructure scaling
- **Post-mortem Analysis**: Reproducible reports for compliance and auditing

Security Operations

- **DDoS Detection**: Identify abnormal traffic from specific IPs
- **Threat Hunting**: Pattern matching for attack signatures
- **Access Control**: Audit who accessed what and when

Business Intelligence

- **User Analytics**: Session analysis and engagement metrics
- **A/B Testing**: Compare traffic patterns across experiments
- **Performance Optimization**: Identify slow endpoints and bottlenecks

5.4 Comparison with Imperative Approaches

Aspect	**Functional (Haskell)**	**Imperative (Python/Java)**
Thread Safety	Guaranteed by immutability	Manual synchronization required
Parallel Scaling	Automatic with parallel strategies	Manual thread management
Testing	Pure functions easily testable	Side effects complicate testing
Debugging	No unexpected state changes	Mutable state hard to track
Correctness	Type system prevents many bugs	Runtime errors common
Refactoring	Safe due to type guarantees	Risky, requires extensive testing